

SEUPD@CLEF: Team retrix on Source Retrieval for Scientific Web Claims

Fabio Baldan¹, Davide Donati¹, Alvisè Garberino¹, Giancarlo Padoan¹ and Marco Tessari¹

¹University of Padua, Italy

Abstract

We present the results of the retrix group for the CheckThat! 2026 - Task 1, in which the goal is to identify the scientific paper implicitly referenced in a social media post. The input consists of short, noisy and usually informal posts that mention scientific findings without providing explicit links or explicit citations. In our pipeline, first we translate the non-English posts, then we expand them using an embedding model and the terms of the top 5 documents (ranked by similarity). After that, the first research is an hybrid retrieval system that combines dense FAISS search with sparse lexical matching and ColBERT multi-vector scoring ($s = 0.5 s_{\text{dense}} + 0.15 s_{\text{sparse}} + 0.35 s_{\text{colbert}}$). Finally a neural re-ranking with a fine-tuned cross-encoder model is performed.

Keywords

Fact Checking, Information Retrieval, Scientific Document Retrieval, Social media fact verification, CLEF, Neural Re-ranking, Cross Encoder

1. Introduction

The rapid spread of scientific claims through social media creates a difficult fact-checking scenario: posts often report, simplify or sensationalize research findings while omitting the original source. In many cases, the cited study is not linked explicitly with a URL or a bibliographic reference but only mentioned indirectly through fragments such as the reported finding, the venue, the institution or the authors. As a consequence, verifying such claims requires first identifying the scientific publication that is being implicitly referenced.

This problem is the focus of Task 1 of the CLEF 2026 CheckThat! Lab, *Source Retrieval for Scientific Web Claims*. Given a social media post containing a scientific claim and an implicit reference to a paper, the goal is to retrieve the correct publication from a fixed collection of 10,000 candidate papers written in English.

The task is challenging for several reasons. First, the linguistic gap between social media and scientific writing is substantial: posts are short, informal and often contain paraphrases, slogans, hashtags or incomplete statements whereas scientific papers use more precise and technical language. Second, the reference to the source paper is implicit rather than explicit so the retrieval system must infer the connection from partial semantic evidence rather than from direct citations. Third, relevant clues may appear in different forms, such as mentions of a scientific result, a medical outcome, a research institution or a publication venue, requiring the system to combine lexical and semantic evidence effectively.

An additional challenge is multilinguality: the official query sets include English, German and French posts while the candidate pool remains shared across languages.

Team Retrix approaches this problem with a four-stage sequential pipeline.

1. In a preliminary stage we translate to English all the queries written in French and German using the model `utter-project/EuroLLM-9B-Instruct-2512` [1], specifically funded by the EU

“Search Engines”, course at the master degree in “Computer Engineering”, Department of Information Engineering, and at the master degree in “Data Science”, Department of Mathematics “Tullio Levi-Civita”, University of Padua, Italy. Academic Year 2025/2026

✉ fabio.baldan.1@studenti.unipd.it (F. Baldan); davide.donati@studenti.unipd.it (D. Donati);
alvisè.garberino@studenti.unipd.it (A. Garberino); giancarlo.padoan@studenti.unipd.it (G. Padoan);
marco.tessari.8@studenti.unipd.it (M. Tessari)



© 2026 Copyright for this paper by its authors.
Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

for understanding and generating text in all European languages. This allows us to handle all queries the same way, independently of the original language and consequently of the semantic and structural differences.

2. Raw tweet-style queries are cleaned of emojis and symbols like "#" and "@" and then enriched via pseudo-relevance feedback: BAAI/bge-large-en-v1.5 [2] encodes both queries and corpus documents into a shared embedding space, retrieves the top-5 semantically similar papers per query and extracts the most frequent terms from those papers as expansion tokens.
3. The expanded queries are fed into BAAI/bge-m3 [3] operating in hybrid mode, which combines dense inner-product search via FAISS [4] with sparse lexical matching and ColBERT multi-vector scoring to produce an initial ranked list of candidates.
4. Finally, the top candidates are re-ranked by a neural cross-encoder, in our case nvidia/llama-nemotron-rerank-1b-v2, fine-tuned on the English training dataset of the CLEF CheckThat! Task 1 2026 and all the queries' datasets of the CLEF CheckThat! SubTask 4b 2025.

The paper is organized as follows: Section 2 discusses the most relevant research directions related to this task; Section 4 describes in more detail our approach; Section 5 explains our experimental setup; Section 6 discusses our main findings; finally, Section 7 draws some conclusions and outlooks for future work.

2. Related Work

The task addressed in this paper lies at the intersection of fact-checking and information retrieval. A natural starting point is lexical retrieval where probabilistic ranking models such as BM25 remain strong baselines thanks to their efficiency, interpretability and robustness in candidate generation [5]. In the setting of scientific claim source retrieval, lexical matching is especially useful when a post mentions recognizable entities or terminology that also appears in the title, abstract, venue or author list of the referenced paper.

Neural retrieval methods have been proposed to mitigate the lexical mismatch between queries and documents by learning dense semantic representations. More broadly, neural information retrieval has shifted attention from exact token overlap to representation learning and learned matching functions [6]. In particular, dense dual-encoder architectures such as Dense Passage Retrieval (DPR) demonstrated that learned query and document embeddings can be highly effective for first-stage retrieval [7]. This line of work is especially relevant to our task because social media posts often paraphrase scientific findings instead of reusing the exact wording found in the original publication. BGE-M3 unified dense, sparse, and multi-vector retrieval in a single multilingual model [8], enabling hybrid scoring that combines embedding similarity with lexical token weights a combination particularly well-suited to cross-lingual settings where queries and documents share few surface tokens.

Vocabulary mismatch between short, informal queries and longer scientific documents can be further reduced through query expansion. Classical pseudo-relevance feedback methods such as RM3 [9] re-weight query terms using top-ranked documents from an initial retrieval pass. Neural extensions of this paradigm apply dense representations to the same idea [10]. In our pipeline, we incorporate embedding-based query expansion to enrich query representations before scoring against the corpus.

Between initial retrieval and final ranking, many modern systems adopt multi-stage pipelines that trade off efficiency and effectiveness [6]. Late-interaction models such as ColBERT preserve fine-grained token-level matching while remaining substantially more scalable than fully pairwise neural scoring over the whole collection [11]. Likewise, transformer-based re-rankers have become a common second-stage component for refining an initial candidate set with stronger query-document interactions [12]. These approaches motivate retrieval architectures that first generate a manageable set of candidates and then apply more expensive relevance estimation only where it is most useful.

Recent work has argued that reranker fine-tuning benefits from hard negatives because they force the model to learn finer-grained relevance boundaries than easy or random negatives provide [13]. A practical instantiation of this strategy trains the reranker on negatives mined from the same retrieval

model whose errors the reranker is meant to correct, a form of bootstrap self-improvement. Adapting large language models to retrieval tasks is further facilitated by parameter-efficient fine-tuning with LoRA [14], which has been shown to match or approach full fine-tuning quality for passage reranking while dramatically reducing GPU memory requirements [15]. This perspective is consistent with recent practical guidance for training cross-encoder rerankers with Sentence Transformers [16]. In that setting, fast embedding-based retrieval is used to build a candidate pool, while the cross-encoder jointly scores query-document pairs and benefits from negatives mined from nearby embedding neighborhoods rather than from trivial mismatches.

Our work is also related to the broader literature on automatic claim verification and more specifically to the CheckThat! shared tasks, which have addressed the detection, verification, and analysis of claims in social media across several editions [17, 18, 19]. Recent participants in the same task line have independently converged on hybrid retrieval plus reranking as the dominant pipeline configuration [20, 21], confirming the high-level design choices taken.

3. Dataset

The dataset consisted of 10,000 scientific publication that had to be paired with tweets-like posts written in three languages: German, French and English. There aren't explicit references such as URLs in any query revised but any tweet contains implicit references to a scientific paper such as mention of the authors or key-words related to the paper. The datasets files are all in JSON format. The publications' dataset contains the columns pubkey (unique identifier), title, abstract, venue / location and authors while the queries' datasets contain the columns index, pubkey (ID of the positive scientific paper that has to be matched with the query) and text.

Table 1
Dataset statistics by language

Split	Language	File	Queries		Length (tokens)			
			#	%	Min	Median	Max	Mean
Train	EN	en_train.json	14977	77.8	2	33	57	31.5
	FR	fr_train.json	2807	14.6	2	33	57	31.5
	DE	de_train.json	1460	7.6	2	33	57	31.5
Total			19244	100	2	33	57	31.5
Dev	EN	en_dev.json	3905	78.2	5	33	60	31.7
	FR	fr_dev.json	702	14.1	5	33	60	31.7
	DE	de_dev.json	386	7.7	5	33	60	31.7
Total			4993	100	5	33	60	31.7
Test	EN	final_en_test.json	6076	74.4	5	33	58	31.9
	FR	final_fr_test.json	1220	14.9	5	33	58	31.9
	DE	final_de_test.json	876	10.7	5	33	58	31.9
Total			8172	100	5	33	58	31.9
Overall Total			32409		2	33	60	31.7

Note: Percentages are computed within each type of datasets (e.g. train percentages are computed between all the training datasets and so on).

Figure 1 compares lengths distribution of tokens across train, dev and test datasets. All three datasets share similar distributions, with the same median value and similar length value peaks.

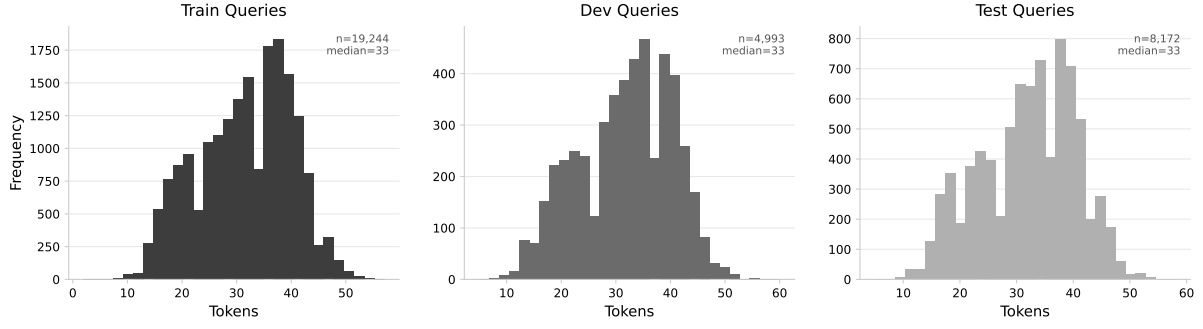


Figure 1: Train, dev and test datasets’ distribution of query token length.

4. Methodology

We started from a lexical baseline based on Apache Lucene BM25 and progressively moved to a multi-stage neural pipeline. Early experiments focused on analyzer choices, stopwords lists and BM25 variants; these runs were useful to establish a stable baseline, but they also showed that lexical overlap alone is insufficient for this task.

We adopted a four-stage pipeline (Figure 2) for the proposed information retrieval system, consisting of:

- **Query Translation:** queries in French or German are first translated into English.
- **Query expansion:** we expanded the English query with additional context to bridge the gap between informal and domain specific language.
- **Retrieval:** we retrieved relevant documents from the corpus using the expanded query.
- **Reranking:** we reranked the retrieved documents based on their relevance to the original query.

For each of these stages, we experimented with different models, choosing the ones that performed better in relation to the others.

In the following sections all the four stages are precisely explained.

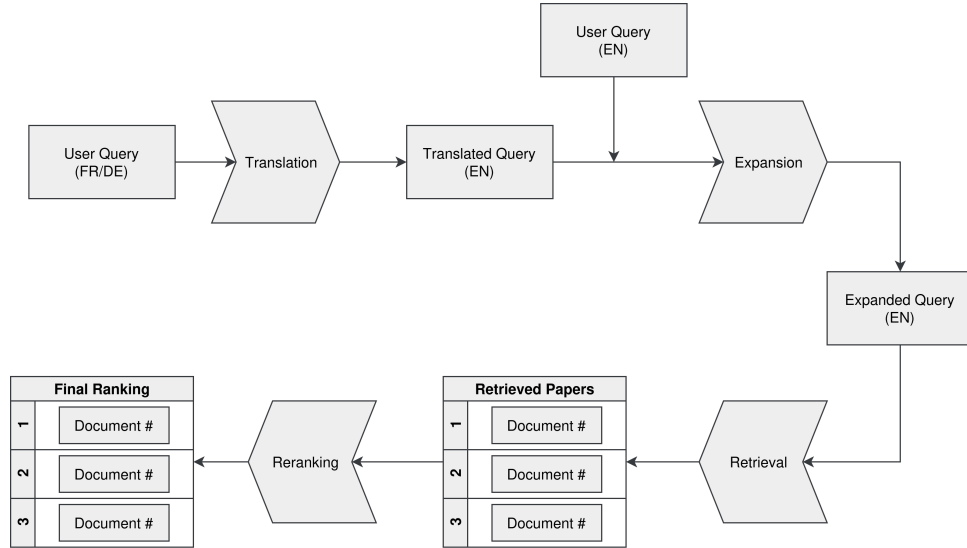


Figure 2: Four-stage pipeline for information retrieval.

4.1. Query translation

In the preliminary stage of our pipeline non-English queries (French and German) are translated into English. This is done query by query using the model `utter-project/EuroLLM-9B-Instruct-2512`

with the following prompt:

LLM Prompt

```
system: You are a professional translator.
Translate the user's text to English.
Output ONLY the English translation.
Do not include the original text, explanations, labels, or any other content.

user: Translate this {French/German} text to English.
Output only the English translation, nothing else.

{Query original text  $x$ }
```

Given x as the original query text, the model predict probabilities for the next token with [22]:

$$P(y_t = i \mid y_{<t}, x) = \text{softmax}(z_t/T)_i$$

Looking at our model configuration, the prediction becomes greedy, following [23]:

$$y_t = \arg \max_i P(y_t = i \mid y_{<t}, x)$$

The resulting English queries are then fed into the following step of our pipeline.

4.2. Query Expansion

Following the translation stage, each query is processed by a deterministic query-expansion module designed to reduce query–document vocabulary mismatch between short user claims and the terminology used in the scientific corpus. The module does not rely on generative LLM-based paraphrasing. Instead, it combines lexical feature extraction with dense semantic retrieval and neighborhood-based term induction. This design is consistent with the established role of query expansion in mitigating vocabulary mismatch in information retrieval and with the use of embedding models for semantic search.

- The translated English query is first analyzed with spaCy’s `en_core_web_sm` pipeline, which provides token-level linguistic annotations, including stop-word indicators and coarse-grained part-of-speech labels from the Universal POS tag set. Candidate lexical anchors are then identified by retaining tokens tagged as NOUN or PROPEN while excluding stop words and tokens shorter than three characters. The resulting set is deduplicated to produce a compact lexical signature that captures the principal nominal content of the query.
- The semantic expansion stage operates over a corpus in which each document is represented as the concatenation of its title and abstract. These title–abstract pairs are embedded with BAAI/bge-large-en-v1.5. In accordance with the documented BGE retrieval interface, query embeddings are produced by prepending the instruction `{Represent this sentence for searching relevant passages: }` to the translated query, whereas the corpus documents are encoded as retrieval targets. Similarity is then computed on normalized embeddings using the dot product. For unit-length vectors, dot-product scoring is equivalent to cosine similarity while being computationally more efficient.

$$s(q, d) = e_q^\top e_d = \sum_i e_{q,i} e_{d,i}$$

Because embeddings $e = \frac{v}{\|v\|_2}$ are L2-normalized $e_q^\top e_d = \cos(q, d)$ [24].

The five highest-scoring documents are retained as the local semantic neighborhood used for expansion.

In the described implementation, candidate expansion terms are extracted from the retrieved documents after lowercasing, ASCII normalization, mention and hashtag removal, stop-word

filtering, and exclusion of short tokens. The candidate pool is aggregated across the retrieved neighborhood and assigned coarse weights derived from the semantic similarity of the source documents, so that terms originating from more similar neighbors contribute more strongly. Terms already present in the query are removed and the fifteen highest-ranking candidates are retained as explicit expansion terms.

$$K = \text{unique}\{\ell(t) : POS(t) \in \{\text{NOUN}, \text{PROPN}\}, \neg stop(t), |t| > 2\}$$

$$w_h = \max(1, \text{round}(5s_h))$$

$$c(t) = \sum_h w_h tf(t, d_h)$$

The final expanded query is constructed by concatenating three components: the cleaned translated query, the noun and proper-noun-based keyword set extracted directly from the query and the expansion terms induced from the retrieved document neighbourhood. This output expanded query is then used by the downstream retrieval pipeline.

Before confirming this as query expansion strategy, we have also tried to use two LLM models, meta-llama/Llama-3.2-1B and openai/gpt-oss-20b with the following prompt:

LLM Prompt

```
system: "Reasoning: {reasoning_effort}"
You are a strict JSON-only retrieval query expansion generator.
Return the final answer only as valid JSON.

user: You generate retrieval-specific query expansions.
Return only one valid JSON object with exactly these string fields:
{
  "keywords": "...",
  "sparse": "...",
  "embedding": "...",
  "colbert": "...",
  "expanded": "..."
}

Rules:
- Do not include markdown, code fences, emoji, commentary, arrays, or nested objects.
- All five values must be strings.
- Do not hallucinate facts.
- Do not translate unless translation is already present in the query.
- Preserve original-language terms.
- keywords: compact important terms only; entities, nouns, technical terms, names, locations, dates, identifiers, acronyms, modifiers; no sentence.
- sparse: lexical BM25/SPLADE-style expansion; exact-match terms, synonyms, acronyms, aliases, spelling and morphology variants; phrase fragments separated by spaces or semicolons; no long prose.
- embedding: concise HyDE/query2doc-style natural-language semantic pseudo-document; include intent, context, and likely relevant-document framing.
- colbert: concise late-interaction expansion; preserve important tokens and entities; add focused context and aliases; avoid synonym spam.
- expanded: readable general-purpose merged expansion string combining the useful hints from the other fields.
```

This approach, which used a different expansion for each retrieval representation, did not improve performance. We therefore retained the simpler and more effective configuration based on a single

expanded query. Section 4.3 describes in more detail how these expansion strategies were evaluated during retrieval.

4.3. Retrieval

The retrieval stage is designed as a high-recall candidate-generation step between query expansion and neural reranking. Its objective is not to make the final relevance decision but to ensure that the true source paper is included in a sufficiently large candidate set for the subsequent, more computationally expensive cross-encoder stage. For this reason, after experimenting with a Lucene BM25 baseline, BGE-M3 using dense vectors only and other bi-encoder models, the final pipeline adopts **BAAI/bge-m3** in a hybrid retrieval setting.

This choice reflects the nature of the task. Social media claims often contain informal paraphrases, hashtags, named entities, numerical values and fragments of scientific terminology. Therefore, relying exclusively on exact lexical overlap or exclusively on dense semantic similarity may discard useful evidence. Hybrid retrieval is better suited to this setting because it combines complementary retrieval signals and can offer higher accuracy and stronger generalization capabilities [25].

The retriever searches the fixed paper collection provided by the task organizers. Each paper is represented at document level by concatenating its title and abstract and the retrieved unit is always the paper pubkey. We did not split papers into smaller chunks because the task requires identifying the source publication rather than retrieving a specific evidence passage. Using one representation per paper also avoids an additional chunk-to-document aggregation step. Although the collection includes metadata such as venue and authors, this hybrid retrieval stage uses only titles and abstracts to speed up the inference time.

All input queries are converted into plain English, either because they are originally written in English or because they are translated before retrieval, see Section 4.1. They are then expanded using the procedure described in Sections 4.2.

We chose **BAAI/bge-m3** because it is well suited to this retrieval setting. The model can produce dense vectors, sparse lexical weights and ColBERT-style multi-vector representations from the same encoder.

This allows the retriever to combine broad semantic matching, exact term evidence and fine-grained token-level matching without switching between unrelated encoders.

For the dense representation, the document or query embedding is obtained from the normalized CLS representation [26]:

$$e = \text{norm}(H[0])$$

For the sparse representation, BGE-M3 assigns lexical weights to tokens [27]:

$$w_{qt} = \text{ReLU}(W_{lex}^T H_q[i])$$

The sparse similarity between a query and a paper is then computed over overlapping terms:

$$s_{lex} = \sum_{t \in q \cap p} w_{qt} w_{pt}$$

Candidate selection is performed in two steps. First, all documents are encoded as dense BGE-M3 vectors and searched with an inner-product FAISS index [4]. The dense FAISS score is computed using inner product [28]:

$$s_{IP}(q, d) = q^T d$$

Since the dense vectors are normalized, this inner product is equivalent to cosine similarity:

$$q^T d = \cos(q, d)$$

This dense stage acts as an efficient broad filter over the whole collection. The default setting retrieves a large candidate pool of `candidate_multiplier × top_k`, with `candidate_multiplier = 3` and

`top_k` = 2000, before the final truncation. The number of candidates passed to the hybrid rescoring stage is:

$$K_c = \min(N, \max(K, Km))$$

This deliberately large candidate set favours recall, which is more important than precision at this stage.

Second, the dense candidates are rescored using the sparse and ColBERT-style representations. The sparse component rewards important overlapping terms, while the ColBERT-style component captures local token correspondences that may be lost when each document is represented by a single dense vector [11]. The ColBERT-style late-interaction score is computed as [29]:

$$s_{colbert}(q, d) = \frac{1}{N_q} \sum_{i=1}^{N_q} \max_{1 \leq j \leq N_d} q_i^\top d_j$$

The final configuration was selected through a series of development runs. We first tuned the lexical baseline by varying the Lucene analyzer, stop-word lists, stemming strategy and BM25 searcher variants. These experiments improved the classical baseline but also highlighted its limitations for this task: even with query expansion and reranking, BM25-based candidate generation remained more dependent on surface-level lexical overlap than desired.

We then evaluated dense retrieval with several bi-encoder models. Among these, **BAAI/bge-m3** achieved the strongest performance and improved the semantic coverage of the candidate set, particularly for paraphrased and multilingual claims. We also tested BGE-M3 in a hybrid retrieval setting, using a setup that differed slightly from the final configuration. In those experiments, sparse, dense, and ColBERT-style retrieval were performed using dedicated query expansions tailored to each retrieval mode. More specifically, each query was represented through three complementary views: an embedding view for semantic matching, a sparse view for lexical and term-based evidence, and a ColBERT view for fine-grained token-level interaction.

However, the best results in our analysis were obtained with a simpler configuration in which BGE-M3 hybrid retrieval used the same expanded query, described in the previous section, for all three retrieval modes. We then tested several weighting configurations. Among them, the run labelled `bge_m3_hybrid_513` was retained because it provided the strongest recall-oriented trade-off among the retrievers evaluated without reranking. This configuration used weights of 0.5, 0.15 and 0.35 for dense, sparse, and ColBERT scoring, respectively, with a multiplier of 3 and a maximum hybrid input length of 384.

A closely related configuration with a larger ColBERT weight achieved a nearly identical `MRR@5` but a lower `Recall@100`. We therefore preferred the configuration that better matched the role of retrieval as a candidate generator. This choice was further confirmed by downstream experiments: reranking candidates from `bge_m3_hybrid_513` consistently outperformed the corresponding BM25-based reranked runs. In addition, increasing the reranked candidate depth from a few hundred documents to between 1000 and 2500 documents permitted to increase the final recall by serving to the neural reranker a bigger candidate-pool to rerank.

We also experimented with a learned fusion strategy for the hybrid signals. In particular, the experimental setup included a LightGBM LambdaMART option that builds features from the dense, sparse, and ColBERT scores, together with normalized score variants and dense-rank features, and then learns how to combine them. However, the retained retrieval configuration uses simpler linear fusion, which is easier to reproduce and aligns well with the recall-oriented role of this stage.

The final hybrid retrieval score is therefore computed as:

$$s(q, d) = 0.5 s_{dense}(q, d) + 0.15 s_{sparse}(q, d) + 0.35 s_{colbert}(q, d).$$

The largest weight is assigned to the dense score because it provides the main semantic signal. The sparse and ColBERT components add complementary lexical and fine-grained matching evidence. Candidates are sorted by this score in descending order and truncated to the final `top_k`. The retrieval

output is a JSON mapping from each query ID to an ordered list of paper pubkeys. Scores are not exported, because the next stage treats retrieval as a candidate generator and recomputes relevance using a cross-encoder over the returned document identifiers.

4.4. Reranking

The reranking stage receives the candidate list produced by the hybrid retriever and reorders it using cross-encoder models. Its goal is to improve early precision, especially MRR@5, while preserving as much recall as possible from the first stage.

Cross-encoder scoring

Our strongest single-stage reranker is `nvidia/llama-nemotron-rerank-1b-v2`, a 1B-parameter LLaMA-based model fine-tuned as a sequence classifier. For each query-document pair, we build an input sequence using the official NVIDIA prompt template:

LLM Prompt

```
question:{query} \n \n passage:{passage}
```

The model computes a scalar logit $z \in \mathbb{R}$ through a linear classification head applied to the final hidden state, and the relevance score is obtained by applying a sigmoid:

$$s(q, d) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Documents are then sorted in descending order of $s(q, d)$. In practice, reranking deeper candidate pools (500–2500 documents) consistently produced better final results than shallower pools, confirming the importance of high recall at the retrieval stage to compensate for miss-labeling from weaker previous retrieval-stages. At this stage, queries are evaluated against the corpus of the documents taking into account their title, abstract and author.

We also evaluated alternative rerankers, including Qwen3 and GTE-ModernBERT variants

4.5. Fine-tuning the Reranker

To improve the Nemotron reranker on the domain-specific characteristics of the CheckThat! task, we fine-tuned `nvidia/llama-nemotron-rerank-1b-v2` using Parameter-Efficient Fine-Tuning (PEFT) with Low-Rank Adaptation (LoRA) on data from both the 2025 and 2026 editions of the task.

Training data and hard-negative mining

Training examples are query-document pairs with binary labels: a positive document is the gold source paper associated with the query, while negative documents are hard negatives mined automatically using `sentence-transformers/static-retrieval-mrl-en-v1`. Hard negatives are selected from the range $[5, 50]$ nearest neighbors (by embedding similarity) that satisfy score ≤ 0.85 and lie at least 0.05 below the positive score in cosine similarity (absolute margin) preventing trivially easy or falsely relevant negatives from polluting training that would under-mine the effectiveness of the fine-tuning.

Training data combines two years of the task:

- **2026 data:** split 90% training / 10% development. The development set is drawn exclusively from 2026 data for consistency with the official CT26 Task 1 evaluation protocol. Due to time-constraints we used just the `en_train` datasets but we suppose that also German and French translated queries datasets could improve the quality of the fine-tuning;
- **2025 data:** all available topics (dev, test-gold, and train splits) are used entirely for training, never for development.

This design ensures that the dev set used to track MRR@5 during training is distribution-aligned with the contest test queries. Each training record contains one query, one positive passage and up to five hard negatives.

LoRA architecture

To fine-tune the 1B-parameter base model with limited GPU memory, we apply LoRA [30] with rank $r = 16$ and scaling factor $\alpha = 32$. Adapters are inserted into all attention and feed-forward projection layers: `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, and `down_proj`. The classification head (`score`) is included in `modules_to_save` and is fully updated during training. The task type is `SEQ_CLS`, matching the sequence-classification objective of the reranker.

Training objective

The model produces a single scalar logit z per input pair. Training minimizes binary cross-entropy with logits over all positive ($y = 1$) and negative ($y = 0$) pairs in each batch:

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^B \left[y_i \log \sigma(z_i) + (1 - y_i) \log(1 - \sigma(z_i)) \right]$$

where B is the effective batch size and σ is the sigmoid function. Gradient accumulation over 4 steps is used to simulate a larger effective batch size of 32 without additional memory overhead.

Optimization

The model is trained with AdamW, a learning rate of 2×10^{-5} , a linear warmup over 10% of the total training steps, and gradient clipping at norm 1.0. Training runs for 3 epochs with bfloat16 mixed precision and gradient checkpointing enabled to reduce VRAM usage. The checkpoint with the best MRR@5 on the 2026 development set is retained.

Inference

At inference time, the LoRA adapter weights are merged into the base model via `merge_and_unload()` before scoring, so no PEFT overhead is incurred during evaluation. The scoring procedure and prompt format are identical to those of the base reranker (Section 4.4) and the fine-tuned model is used as a drop-in replacement in the reranking stage.

4.6. Evaluation

We evaluate systems on the CheckThat! Task 1 query sets by matching retrieved pubkeys against the gold pubkey for each query. The official optimization target is MRR@5, which we use as the primary score during model and parameter selection. In parallel, we track Recall@100 to monitor candidate-generation quality and to verify that reranking gains do not come from overly aggressive filtering.

For internal analysis, our evaluator also computes additional ranking metrics (e.g., Recall@1/5/10/100, Precision@1/5/10, MAP, and nDCG@k). This broader view is useful to interpret trade-offs between recall-oriented retrieval settings and precision-oriented reranking settings while keeping MRR@5 as the main criterion for reporting and model selection.

5. Experimental Setup

All experiments were conducted on our machines, utilizing a computer with an AMD Ryzen 9 5900X, 32 GB RAM and NVIDIA RTX 3090 GPU (10496 CUDA Cores, boost clock 1.70 GHz, 24 GB of GDDR6X).

memory, 35.58 TFlops) for resource-intensive workload and our laptops for standard workloads and refining of the programs. We have four laptops, the two most powerful are a MacBook Air M3 and one with no dedicated GPU, 16GB RAM and a Intel core i7-13700H. For reproducibility, all the environment settings have been documented and are freely available or easily obtained by following the instructions in this paper.

6. Results and Discussion

6.1. Main Results

Table 2 summarizes representative configurations from the run-analysis sheets performed. The table is not meant to list every run we have made, instead, it selects runs that cover the main experimental families: analyzer tuning, hybrid retrieval, single-stage neural reranking and alternative rerankers. The comparison uses the English run-analysis sheet with 14,977 evaluated queries, the ones of the `train_set` and all metric values are rounded to three decimals.

In the analyzer rows, `MyEnglishAnalyzer_V2` denotes the Lucene analyzer using the RankSNL large stopword list, ASCII folding, lowercasing, English possessive removal and KStem. Searcher V2 denotes the BM25 searcher variant that adds Boolean SHOULD clauses.

Table 2
Representative English run-analysis results performed on `en_train_set`.

Family	Run label	Key configuration	MRR@5	Recall@100
Analyzer tuning	Analyzer V2	BM25 retrieval with <code>MyEnglishAnalyzer_V2</code> , Searcher V2, and BGE-V1 query expansion	0.503	0.796
Analyzer tuning	Analyzer V1.12	BM25 retrieval with KStem and an EBSCO-host/MEDLINE/CINAHL stoplist variant	0.450	0.783
Retrieval only	Hybrid A	BGE-M3 hybrid retrieval with dense, sparse, and ColBERT weights of 0.35, 0.15, and 0.50	0.557	0.837
Retrieval only	Hybrid B	BGE-M3 hybrid retrieval with dense, sparse, and ColBERT weights of 0.50, 0.15, and 0.35	0.557	0.839
Best model reranking	Nemotron top-400	<code>bge_m3_hybrid_513</code> candidates reranked with <code>nvidia/llama-nemotron-rerank-1b-v2</code>	0.694	0.893
Best model reranking	Nemotron top-1000	<code>bge_m3_hybrid_513</code> candidates reranked with <code>nvidia/llama-nemotron-rerank-1b-v2</code>	0.702	0.911
Alternative rerankers	Qwen3-4B	<code>bge_m3_hybrid_513</code> candidates reranked with Qwen3-reranker-4B	0.662	0.839
Alternative rerankers	GTE-ModernBERT	<code>bge_m3_hybrid_513</code> candidates reranked with Alibaba-NLP/gte-reranker-modernbert-base	0.652	0.852

The results show a consistent progression from lexical retrieval toward neural ranking. The best retrieval-only configuration reaches an MRR@5 of 0.557, while the strongest reranked run reported in Table 2 reaches 0.702. The largest observed step comes from adding `nvidia/llama-nemotron-rerank-1b-v2`: the top-1000 Nemotron run reaches an MRR@5 of 0.702 and Recall@100 of 0.911, compared with 0.557 and 0.839 for the retained hybrid retriever. We do not have traced and reported confidence intervals, paired significance tests, or effect sizes, so these differences are discussed as observed metric differences only. Due to limited computing resources and time constraints we have not been able to perform a meaningful run on the english training dataset: it would have demanded more than 40 hours of computation on our RTX 3090 for a run on the top-k=1000 reranking).

For the runs performed in the `dev_set` we provide a separate multilingual check rather than a single aggregate score. On the English development split, the non-fine-tuned Nemotron baseline reaches 0.675 MRR@5 and 0.835 Recall@100 when reranking the top 100 candidates; 0.692 MRR@5 and 0.909 Recall@100 when reranking the top 1000 candidates. The fine-tuned Nemotron LoRA run on the same English development split reaches 0.716 MRR@5 and 0.913 Recall@100. The corresponding fine-tuned

development runs reach 0.76 MRR@5 and 0.929 Recall@100 for French and 0.67 MRR@5 and 0.889 Recall@100 for German with the top-2000 candidate setting.

6.2. Ablation / Run Analysis

Analyzer tuning: We initially explored stronger English analyzers and several stemming and stop-word variants in order to improve lexical matching between short social-media posts and scientific abstracts. This family peaks at an MRR@5 of 0.503 with `MyEnglishAnalyzer_v2`, while the other analyzer variants remain around the 0.45-0.50 range. These runs were useful in validating the pre-processing choices but they also showed that text normalization alone is not enough to bridge the semantic gap between informal claims and academic documents. Analyzer tuning yields measurable improvements but its impact is limited when compared with semantic retrieval and neural reranking.

Hybrid retrieval without reranking: A shift from analyzer-focused experiments to hybrid retrieval leads to a substantially larger improvement. The two best retrieval-only systems, reported in Table 2 as Hybrid A and Hybrid B, both reach an MRR@5 of about 0.557 with recall@100 between 0.837 and 0.839. The difference between them lies in the balance between the dense and ColBERT components while the sparse contribution is kept fixed. This indicates that combining dense, sparse and ColBERT-style signals already produces a strong candidate set before any reranking is applied. At the same time the very small difference between the two configurations suggests that, once the hybrid formulation is in place, the exact internal weighting matters less than the transition away from purely lexical retrieval. Hybrid retrieval is the first major step change in effectiveness and provides the strongest retrieval-only foundation for the full pipeline.

Single-stage reranking with Nemotron: Adding `nvidia/llama-nemotron-rerank-1b-v2` on top of the hybrid candidates produces the largest isolated gain in MRR@5. The representative Nemotron runs increase from the 0.557 range of the retrieval-only systems to 0.694 and 0.702, while recall@100 also rises from about 0.839 to as high as 0.911. We also observe a limited but consistent benefit from increasing the number of candidates exposed to the reranker: moving from the top-400 to the top-1000 setting slightly improves both MRR@5 and recall@100. This suggests that the reranker can exploit a broader candidate pool enduring the additional noise bringing further, although diminishing, gains. In our experiments, the threshold at which the MRR no longer showed signs of improvement was around `topk=2500`; however, it is also true that it is unlikely to find many more relevant documents that are easy to detect at those depths.

Alternative rerankers: To verify that the gains were not tied to a single model, we also evaluated other rerankers, including `Qwen/Qwen3-Reranker-4B` and `Alibaba-NLP/gte-reranker-modernbert-base`. Both models clearly outperform the retrieval-only baseline in terms of MRR@5, reaching values of 0.662 and 0.652, respectively. However, they remain below the Nemotron runs, especially on MRR@5, which suggests weaker ranking quality in the first positions of the final list. Even so these experiments confirm that the hybrid retriever produces candidate sets that can be improved by different neural rerankers. Alternative rerankers validate the overall pipeline design but Nemotron is the strongest first-stage reranker among the models we tested.

7. Conclusions and Future Work

The results of the described system support three main conclusions.

First, analyzer and BM25 tuning provide a useful lexical baseline, especially thanks to its light computing print, with the best analyzer-focused run reaching 0.503 MRR@5 and 0.796 Recall@100 in the English run analysis.

Second, moving to BGE-M3 hybrid retrieval produces a stronger candidate generator: the retained configuration, `bge_m3_hybrid_513`, reaches 0.557 MRR@5 and 0.839 Recall@100.

Third, reranking the hybrid candidates with `nvidia/llama-nemotron-rerank-1b-v2` gives the largest observed improvement among the reported single-stage runs, reaching 0.702 MRR@5 and 0.911 Recall@100 when reranking the top 1000 candidates. Alternative rerankers such as `Qwen3-reranker-4B` and `Alibaba-NLP/gte-reranker-modernbert-base` also improve over retrieval alone, but remain below the Nemotron runs in the reported MRR@5 values.

The development results show that fine-tuning the Nemotron reranker is promising. Fine-tuning improves the performance by a small but appreciable amount (average of +0.03 of MRR@5 in our experiments). Nevertheless we recognize the limits of our knowledge, so a higher-quality fine-tuning that can bring better results is certainly possible.

7.1. Limitations and Future Work

The main limitation of the current evaluation is that the neural stages are computationally demanding: the documented pipeline relies on CUDA-capable hardware for BGE-M3 encoding and cross-encoder reranking, which makes latency and candidate-depth trade-offs an important practical concern.

Future work should therefore focus on a more systematic validation of the current pipeline. First, the reranking and retrieval comparisons should be complemented with paired statistical tests and per-language error analysis. Second, the candidate depth used for Nemotron reranking should be studied together with runtime and memory costs, since deeper reranking improves Recall@100 and MRR@5 in the available runs but increases the amount of neural scoring required. Third, the existing fine-tuning code for rerankers and retrievers should be evaluated more systematically with hard negatives. We were fortunate enough to have access to the CLEF CheckThat! Subtask 4b datasets, which saved us from having to devote some of our resources and time to studying techniques such as data augmentation; however, it is plausible that these techniques could improve the quality of fine-tuning, along with a more in-depth study of fine-tuning techniques using hard negatives. Finally, the translation and query-expansion stages should be analyzed per language, since the current results suggest that the same pipeline can be applied across English, French, and German but do not yet isolate where cross-lingual errors enter the ranking process.

8. Declaration on Generative AI

During the writing of this paper the authors used Grammarly and DeepL for grammar and spelling checks and Claude Sonnet 4.6 for writing-style improvement, citation management and formatting assistance. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the publications content.

9. Group Members Contribution

Fabio Baldan has contributed to this project by:

1. Helping designing and implementing the initial BM25-basic pipeline;
2. Writing and revising some chapters of the documentation based on the actual code and on the concepts learned during the project development;
3. Tracking and reviewing the annotation-part for the runs with various systems;
4. Trying various models for the re-rank part to enhance the MRR@5;
5. Implementing a custom evaluator to focus on metrics that matter the most;
6. Reviewing and improving the query-expansion;
7. Fine-tuning the re-reranker to improve the MRR@5 and assure an advantage if compared with the default one.

Davide Donati has contributed to this project by:

1. Helping designing and implementing the initial BM25 basic pipeline;
2. Implementing a BooleanQuery-based parsing in `Searcher.java` to separate the original tweet and expanded terms into different clauses with tailored boosts (and applying field filters for venue/authors), improving retrieval precision;
3. Implementing and testing the multilingual query-expansion pipeline (e.g. `query_expansion_multilingual.py`, tested with `PROVEDAVIDE.py`) and fixing issues found during integration;
4. Adding a PRF (pseudo-relevance feedback) implementation and accompanying AI-assisted documentation/examples (PRF code is available in the discarded-code area for reference and analysis);
5. Contributing to documentation and write-ups;
6. Keeping track of experimental runs and results for the systems he developed and tested.

Alvise Garberino has contributed to this project by:

1. Helping design and implement the initial BM25-based retrieval pipeline;
2. Helping and searching the best models and techniques to implement a re-ranker model;
3. Implementing and improving the re-ranker fine-tuning pipeline, including data augmentation through back-translation, hard-negative construction/mining, and data-distillation support;
4. Contributing to the evaluation workflow by tracking experimental runs, analyzing results, and supporting checkpoint evaluation;
5. Contributing to the Java support code and documentation, including refactoring/query-model improvements and JavaDoc updates;
6. Writing and revising parts of the documentation, especially the introduction, related work, results, conclusion, and experimental setup sections, while also selecting and integrating the corresponding academic references.

Giancarlo Padoan has contributed to this project by:

1. Designing, implementing and fixing the initial basic pipeline;
2. Writing and correcting some chapters of the documentation based on the actual code and on the notions learned while building the project;
3. Designing and implementing an hybrid-retrieval system to enhance recall, which is the core of the final architecture;
4. Trying various models for the embedding-representation part to enhance the MRR@5;
5. Implementing the first version of query-expansion functionality, and improving it by testing different techniques;
6. Tested different scoring function for combining the hybrid-retrieval scores;
7. Testing various cross-encoder models for the re-ranking stage of the pipeline;
8. Keeping track of the results of the systems that he tried.

Marco Tessari has contributed to this project by:

1. Helping designing and implementing the initial BM25-basic pipeline;
2. Improved the performance of the English Analyzer (text-processing, stemmer, filters and stopwords);
3. Writing part of the docs and helping other members by reviewing their writes;
4. Keeping track of the results of the systems that he tried;
5. Helping and searching the best models and techniques to implement an embedding model;
6. Experimenting with word2vec static embeddings for query expansion;
7. Helping to find the best weights to enhance recall in the hybrid-retrieval system.

References

- [1] Unbabel and Instituto Superior Técnico and Instituto de Telecomunicações and University of Edinburgh and Aveni and University of Paris-Saclay and University of Amsterdam and Naver Labs and Sorbonne Université, EuroLLM-9B-Instruct: A 9B Parameter Multilingual Transformer LLM for European and Relevant Languages, Hugging Face, 2026. URL: <https://huggingface.co/utter-project/EuroLLM-9B-Instruct>, accessed: 07-05-2026.
- [2] BAAI (Beijing Academy of Artificial Intelligence), BGE-large-en-v1.5: A General Embedding Model for English, Hugging Face, 2023. URL: <https://huggingface.co/BAAI/bge-large-en-v1.5>, accessed: 07-05-2026.
- [3] BAAI (Beijing Academy of Artificial Intelligence), BGE-M3: A Multi-Lingual, Multi-Functionality, Multi-Granularity Embedding Model, Hugging Face, 2024. URL: <https://huggingface.co/BAAI/bge-m3>, accessed: 07-05-2026.
- [4] Faiss, Faiss documentation, <https://faiss.ai/>, ????. Accessed: 01-05-2026 (DD-MM-YYYY, italian date format).
- [5] S. E. Robertson, U. Zaragoza, The Probabilistic Relevance Framework: BM25 and Beyond, *Foundations and Trends in Information Retrieval (FnTIR)* 3 (2009) 333–389.
- [6] S. Marchesin, A. Purpura, G. Silvello, Focal elements of neural information retrieval models. An outlook through a reproducibility study, *Information Processing & Management* 57 (2020) 102109:1–102109:29.
- [7] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, W.-t. Yih, Dense Passage Retrieval for Open-Domain Question Answering, in: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Association for Computational Linguistics, Online, 2020, pp. 6769–6781. URL: <https://aclanthology.org/2020.emnlp-main.550/>. doi:10.18653/v1/2020.emnlp-main.550.
- [8] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, Z. Liu, M3-Embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation, in: *Findings of the Association for Computational Linguistics: ACL 2024*, Association for Computational Linguistics, Bangkok, Thailand, 2024, pp. 2318–2335. doi:10.18653/v1/2024.findings-acl.137.
- [9] V. Lavrenko, W. B. Croft, Relevance based language models, in: *Proceedings of the 24th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, ACM, New Orleans, LA, USA, 2001, pp. 120–127. doi:10.1145/383952.383972.
- [10] X. Wang, C. Macdonald, N. Tonellotto, I. Ounis, ANCE-PRF: Indexing disentanglement for pseudo-relevance feedback with approximate nearest neighbor search, in: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 2021, pp. 5765–5778. doi:10.18653/v1/2021.emnlp-main.453.
- [11] O. Khattab, M. Zaharia, ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT, in: *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2020)*, ACM Press, New York, USA, 2020, pp. 39–48. URL: <https://doi.org/10.1145/3397271.3401075>. doi:10.1145/3397271.3401075.
- [12] R. Nogueira, Z. Jiang, R. Pradeep, J. Lin, Document Ranking with a Pretrained Sequence-to-Sequence Model, in: *Findings of the Association for Computational Linguistics: EMNLP 2020*, Association for Computational Linguistics, Online, 2020, pp. 708–718. URL: <https://aclanthology.org/2020.findings-emnlp.63/>. doi:10.18653/v1/2020.findings-emnlp.63.
- [13] H. Meghwani, A. Agarwal, P. Pattnayak, H. L. Patel, S. Panda, Hard Negative Mining for Domain-Specific Retrieval in Enterprise Systems, <https://arxiv.org/abs/2505.18366>, 2025. arXiv:2505.18366, accessed: 06-05-2026 (DD-MM-YYYY, italian date format).
- [14] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, LoRA: Low-rank adaptation of large language models, in: *International Conference on Learning Representations (ICLR)*, 2022. URL: <https://openreview.net/forum?id=nZeVKeeFYf9>.

- [15] A. Nijasure, T. Chowdhury, S. Ghodrathnama, How relevance emerges: Interpreting LoRA fine-tuning in reranking llms, arXiv preprint arXiv:2504.08780 (2025). [arXiv:2504.08780](https://arxiv.org/abs/2504.08780).
- [16] T. Aarsen, Training and Finetuning Reranker Models with Sentence Transformers, <https://huggingface.co/blog/train-reranker>, 2025. Hugging Face blog. Accessed: 06-05-2026 (DD-MM-YYYY, italian date format).
- [17] T. Elsayed, P. Nakov, A. Barrón-Cedeño, M. Hasanain, R. Suwaileh, G. Da San Martino, P. Atanasova, Overview of the CLEF-2019 CheckThat!: Automatic Identification and Verification of Claims, in: F. Crestani, M. Braschler, J. Savoy, A. Rauber, H. Müller, D. E. Losada, G. Heinatz Bürki, L. Cappellato, N. Ferro (Eds.), Experimental IR Meets Multilinguality, Multimodality, and Interaction. Proceedings of the Tenth International Conference of the CLEF Association (CLEF 2019), Lecture Notes in Computer Science (LNCS) 11696, Springer, Heidelberg, Germany, 2019, pp. 301–321.
- [18] A. Barrón-Cedeño, T. Elsayed, P. Nakov, G. Da San Martino, M. Hasanain, R. Suwaileh, F. Haouari, N. Babulkov, B. Hamdan, A. Nikolov, S. Shaar, Z. Sheikh Ali, Overview of CheckThat! 2020: Automatic Identification and Verification of Claims in Social Media, in: A. Arampatzis, E. Kanoulas, T. Tsikrika, S. Vrochidis, H. Joho, C. Lioma, K. Eickhoff, A. Névél, L. Cappellato, N. Ferro (Eds.), Experimental IR Meets Multilinguality, Multimodality, and Interaction. Proceedings of the Eleventh International Conference of the CLEF Association (CLEF 2020), Lecture Notes in Computer Science (LNCS) 12260, Springer, Heidelberg, Germany, 2020, pp. 215–236.
- [19] P. Nakov, G. Da San Martino, T. Elsayed, A. Barrón-Cedeño, R. Míguez, S. Shaar, F. Alam, F. Haouari, M. Hasanain, W. Mansour, B. Hamdan, Z. S. Ali, N. Babulkov, A. Nikolov, G. K. Shahi, J. M. Struß, T. Mandl, M. Kutlu, Y. S. Kartal, Overview of the CLEF-2021 CheckThat! Lab on Detecting Check-Worthy Claims, Previously Fact-Checked Claims, and Fake News, in: K. S. Candan, B. Ionescu, L. Goeuriot, B. Larsen, H. Müller, A. Joly, M. Maistro, F. Piroi, G. Faggioli, N. Ferro (Eds.), Experimental IR Meets Multilinguality, Multimodality, and Interaction. Proceedings of the Twelfth International Conference of the CLEF Association (CLEF 2021), Lecture Notes in Computer Science (LNCS) 12880, Springer, Heidelberg, Germany, 2021, pp. 264–291.
- [20] P. J. Sager, A. Kamaraj, B. F. Grewe, T. Stadelmann, Deep retrieval at CheckThat! 2025: Identifying scientific papers from implicit social media mentions via hybrid retrieval and re-ranking, in: G. Faggioli, N. Ferro, P. Rosso, D. Spina (Eds.), Working Notes of the Conference and Labs of the Evaluation Forum (CLEF 2025), volume 4038 of *CEUR Workshop Proceedings*, CEUR-WS.org, Madrid, Spain, 2025. URL: https://ceur-ws.org/Vol-4038/paper_89.pdf, accessed: 07-05-2026.
- [21] J. Schofield, S. Tian, H. T. T. Truong, M. Heil, DS@GT at CheckThat! 2025: Exploring retrieval and reranking pipelines for scientific claim source retrieval on social media discourse, in: G. Faggioli, N. Ferro, P. Rosso, D. Spina (Eds.), Working Notes of the Conference and Labs of the Evaluation Forum (CLEF 2025), volume 4038 of *CEUR Workshop Proceedings*, CEUR-WS.org, Madrid, Spain, 2025, pp. 1–10. URL: https://ceur-ws.org/Vol-4038/paper_93.pdf, accessed: 07-05-2026.
- [22] Hugging Face, Generation, https://huggingface.co/docs/transformers/main_classes/text_generation, ??? Transformers documentation. Accessed: 2026-05-07.
- [23] UTTER Project, EuroLLM-9B-Instruct-2512, <https://huggingface.co/utter-project/EuroLLM-9B-Instruct-2512>, ??? Hugging Face model card. Accessed: 2026-05-07.
- [24] BAAI, Baai bge-large-en / bge-large-en-v1.5 model card, ??? URL: <https://huggingface.co/BAAI/bge-large-en>.
- [25] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, Z. Liu, Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation, 2024. [arXiv:2402.03216](https://arxiv.org/abs/2402.03216).
- [26] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, Z. Liu, Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation, arXiv preprint arXiv:2402.03216 (2024). URL: <https://arxiv.org/abs/2402.03216>.
- [27] BAAI, Bge-m3 documentation, 2024. URL: https://bge-model.com/bge/bge_m3.html.
- [28] FAISS, Faiss: Metrictype and distances, ??? URL: <https://github.com/facebookresearch/faiss/wiki/MetricType-and-distances>.
- [29] O. Khattab, M. Zaharia, Colbert: Efficient and effective passage search via contextualized late

interaction over bert, in: SIGIR, 2020. URL: <https://arxiv.org/abs/2004.12832>.

- [30] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, Lora: Low-rank adaptation of large language models, in: ICLR, 2022. URL: <https://arxiv.org/abs/2106.09685>.